

Essa é uma consideração crítica. A escolha da arquitetura é fundamental para garantir **escalabilidade, resiliência, manutenibilidade** e, o mais importante, a **viabilidade comercial** do seu software jurídico na nuvem.

Para um software SaaS (*Software as a Service*) de consultoria jurídica inteligente, a arquitetura ideal é o **Microserviços** rodando em **Containerização** (Docker) e orquestrada por **Kubernetes**, seguindo o modelo de **Multi-tenancy**.

---

### 1. Arquitetura Fundamental: Microserviços

A arquitetura de Microserviços é a mais recomendada para sistemas modernos baseados em nuvem (CLOUD), especialmente para um produto que será comercializado e escalado.

#### O que é?

Em vez de um único bloco monolítico, o software é dividido em um conjunto de **serviços menores e independentes** que se comunicam entre si. Cada serviço é responsável por uma funcionalidade específica (ex: um serviço para o CRM, um serviço para a Análise Preditiva, um serviço para a Pesquisa Semântica).

#### Vantagens Chave

- **Escalabilidade Independente:** Se o serviço de "Análise Preditiva" for o mais utilizado, apenas ele pode ser escalado horizontalmente (adicionar mais instâncias) sem afetar o CRM ou o agendamento.
  - **Resiliência (Tolerância a Falhas):** Se um microserviço falhar (ex: o serviço de "Geração de Documentos" cai), o resto do sistema (como o login, o CRM e a pesquisa) continua funcionando normalmente.
  - **Manutenibilidade e Desenvolvimento:** Diferentes equipes podem trabalhar e implantar novos recursos em serviços separados, reduzindo o risco de introduzir bugs em outras partes do sistema.
  - **Escolha Tecnológica (Polyglot):** Você pode usar a linguagem de programação mais adequada para cada tarefa (ex: Python para os modelos de IA, e Java/Node.js para os serviços de *backend* web), otimizando a performance.
-

## 2. Estrutura de Implantação: Containerização (Docker e Kubernetes)

Para executar a arquitetura de microsserviços na CLOUD de forma eficiente e comercializável, você precisa de **containerização**.

- **Docker:** Cada microsserviço é empacotado em um **container Docker**. Isso garante que o serviço funcione de maneira idêntica em qualquer ambiente (desenvolvimento, teste ou produção), eliminando problemas de compatibilidade.
- **Kubernetes (K8s):** É a ferramenta de **orquestração** que gerencia os containers em seu ambiente CLOUD.
  - **Auto-Healing:** Reinicia containers que falham.
  - **Load Balancing:** Distribui o tráfego de usuários de forma uniforme entre as instâncias dos microsserviços.
  - **Escalabilidade Automática:** Adiciona ou remove containers automaticamente com base na demanda de uso.

Essa combinação (Microsserviços + K8s) é o padrão ouro para a construção de um **SaaS (Software as a Service)** escalável.

---

## 3. Modelo de Comercialização: Arquitetura *Multi-tenancy*

A **Arquitetura *Multi-tenancy*** (Múltiplos Inquilinos) é crucial para a viabilidade comercial de um software SaaS na CLOUD.

### O que é?

Significa que **todas as empresas de consultoria (seus clientes)** usam a **mesma instância de software** e a **mesma infraestrutura**, mas seus dados e configurações são **isolados** de forma segura.

| Modelo                                              | Descrição                                                                                                                                                    | Vantagem Comercial                                                                                                                                                                              |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>Multi-tenancy</i></b><br><b>(Recomendado)</b> | Uma única instalação do software e um único banco de dados principal hospedam dados de todos os clientes ( <i>tenants</i> ), com segregação lógica rigorosa. | <b>Custo-Benefício e Escalabilidade:</b><br>Reduz drasticamente o custo por cliente (você não precisa de um servidor e banco de dados para cada novo cliente), facilitando preços competitivos. |

| Modelo                | Descrição                                                               | Vantagem Comercial                                                                               |
|-----------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>Single-tenancy</b> | Uma instalação separada do software e banco de dados para cada cliente. | Mais seguro e personalizável, mas extremamente caro e complexo de gerenciar em escala comercial. |

### Implementação Segura

A segurança dos dados é vital. No *Multi-tenancy*, o software deve garantir:

- 1. Isolamento de Dados:** Cada consulta ao banco de dados deve ser filtrada rigorosamente pela **ID do Tenant** (identificador da empresa). O serviço do CRM da Consultoria A nunca deve conseguir acessar os dados da Consultoria B.
- 2. Configuração por Tenant:** O software permite que cada cliente tenha suas próprias permissões, logotipos e configurações de fluxos de trabalho, tudo armazenado de forma centralizada.

---

### Resumo da Arquitetura Recomendada

| Componente             | Função Principal                                   |
|------------------------|----------------------------------------------------|
| <b>Arquitetura</b>     | <b>Microserviços</b>                               |
| <b>Hospedagem</b>      | <b>Servidor CLOUD</b> (AWS, Google Cloud ou Azure) |
| <b>Implantação</b>     | <b>Docker e Kubernetes (K8s)</b>                   |
| <b>Comercialização</b> | <b>Multi-tenancy</b>                               |

O próximo passo seria definir a **pilha tecnológica** específica para cada microserviço, focando nas ferramentas de IA (bibliotecas de Python) e no banco de dados. Gostaria de focar nas opções de banco de dados para lidar com o volume e a natureza dos dados jurídicos (estruturados e não estruturados)?

---

## Banco de dados

---

A escolha do **Banco de Dados (DB)** é um pilar crucial na arquitetura de microsserviços, especialmente para um software que lida com a diversidade dos dados jurídicos: textos complexos, dados estruturados de clientes e grandes volumes de jurisprudência.

Para a arquitetura de Microsserviços, a melhor abordagem é a **Persistência Poliglota (Polyglot Persistence)**, utilizando diferentes tipos de bancos de dados para atender às necessidades específicas de cada microsserviço.

---

### Persistência Poliglota para Dados Jurídicos

Em vez de forçar todos os dados em um único DB (o que seria ineficiente), usamos o tipo de banco de dados mais adequado para cada tarefa:

#### 1. Banco de Dados Relacional (SQL) – Para Dados Estruturados

O DB relacional é ideal para dados que precisam de alta consistência, transações confiáveis e integridade referencial.

| Microsserviço                | Tipo de Dado                                                     | DB Recomendado                             | Por Que?                                                                                                                                                                                                   |
|------------------------------|------------------------------------------------------------------|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CRM e Faturamento            | Dados de Clientes, Leads, Contratos, Honorários, IDs de Tenants. | PostgreSQL ou MySQL (Gerenciados na CLOUD) | Alta confiabilidade, excelente para transações (ACID) e para garantir o <b>isolamento de dados</b> na arquitetura <i>multi-tenancy</i> . O PostgreSQL tem um excelente suporte a JSON e pesquisa de texto. |
| Gestão Processual (Workflow) | Prazos, Status de Tarefas, Agendamentos.                         | PostgreSQL ou MySQL                        | Necessidade de integridade referencial (uma tarefa pertence a um caso, que pertence a um cliente).                                                                                                         |

---

#### 2. Banco de Dados NoSQL (Não Relacional) – Para Escala e Flexibilidade

Os NoSQLs são essenciais para lidar com o grande volume de dados não estruturados e o desempenho exigido pelas funcionalidades de IA.

##### A. Banco de Dados de Documentos (Document Database)

| Microserviço                         | Tipo de Dado                                                                            | DB Recomendado                                            | Por Que?                                                                                                                                                    |
|--------------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Geração/Revisão de Documentos</b> | <i>Templates</i> de Contratos e Petições, dados semiestruturados de <i>compliance</i> . | <b>MongoDB</b> ou <b>Couchbase</b> (Gerenciados na CLOUD) | Flexibilidade para armazenar documentos jurídicos complexos em formato JSON ou BSON, facilitando a atualização e a busca de cláusulas por campos dinâmicos. |

## B. Banco de Dados de Busca e Análise (Search Database)

| Microserviço                       | Tipo de Dado                                                        | DB Recomendado                             | Por Que?                                                                                                                                                                                            |
|------------------------------------|---------------------------------------------------------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Pesquisa Jurídica Semântica</b> | Jurisprudência, Legislação, Doutrina (Indexação de texto completo). | <b>Elasticsearch</b> ou <b>Apache Solr</b> | São <i>motores</i> de busca otimizados para indexação rápida e pesquisa complexa em grandes volumes de texto não estruturado, essenciais para a busca semântica e a análise de <i>tokens</i> (PLN). |

## 3. Banco de Dados de Chave-Valor (Key-Value Store) – Para Desempenho

Este tipo de DB é usado para aumentar a velocidade do sistema, armazenando dados frequentemente acessados na memória (*caching*).

| Microserviço                 | Tipo de Dado                                                                                         | DB Recomendado                                          | Por Que?                                                                                                                                              |
|------------------------------|------------------------------------------------------------------------------------------------------|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Performance e Caching</b> | Sessões de Usuários, dados do <i>Lead Scoring</i> em tempo real, resultados de consultas frequentes. | <b>Redis</b> ou <b>Memcached</b> (Serviços Gerenciados) | Armazenamento de alta velocidade na memória, reduzindo a latência e a carga sobre os bancos de dados principais, melhorando a experiência do usuário. |

## Conclusão

Adotar a **Persistência Poliglota** garante que você tenha a **ferramenta certa para cada trabalho**: o PostgreSQL para as transações críticas e o Elasticsearch para a inteligência de busca e análise, criando um sistema robusto, rápido e escalável.

A **Segurança da Informação e Conformidade com a LGPD (Lei Geral de Proteção de Dados)** não são apenas requisitos legais, mas sim um diferencial comercial para um software jurídico SaaS.

Para garantir a confiabilidade e a comercialização do seu sistema, devemos focar em três pilares: **Segurança na Arquitetura (*Security by Design*)**, **Conformidade com a LGPD** e **Isolamento de Dados no *Multi-tenancy***.

---

### 1. Segurança na Arquitetura (*Security by Design*)

A arquitetura de microsserviços e nuvem deve ser construída com segurança integrada desde o início.

- **Criptografia Ponta a Ponta (End-to-End Encryption):**
  - **Dados em Repouso (*Data at Rest*):** Todos os bancos de dados (PostgreSQL, Elasticsearch, etc.) devem ter a criptografia de disco ativada (AES-256) na CLOUD. Isso protege os dados mesmo em caso de acesso físico não autorizado aos servidores.
  - **Dados em Trânsito (*Data in Transit*):** Toda a comunicação entre o usuário e o servidor, e entre os microsserviços (APIs internas), deve ser protegida por **TLS/SSL (HTTPS)**.
- **Gestão de Acesso Baseada em Funções (RBAC - Role-Based Access Control):** O sistema deve definir rigorosamente quem pode acessar o quê.
  - **Interno (Consultoria):** Um estagiário deve ter permissão para visualizar dados básicos de casos, mas não para acessar ou editar documentos sigilosos ou informações de faturamento.
  - **Externo (*Tenants*):** As permissões devem ser restritas ao escopo da própria consultoria (isolamento de dados).
- **Monitoramento e Auditoria (Logging):** Todos os acessos, alterações de dados e atividades críticas devem ser registrados. Isso é vital para auditorias de segurança e para demonstrar conformidade com a LGPD.

---

### 2. Conformidade com a LGPD (Dados Pessoais Sensíveis)

O software jurídico lida com dados pessoais e frequentemente com **dados pessoais sensíveis** (ex: questões de saúde em casos trabalhistas, informações financeiras). A LGPD exige cuidados específicos:

- **Anonimização e Pseudonimização:**
  - Para o módulo de **Análise Preditiva**, a IA deve ser treinada em dados que passaram por **pseudonimização** (substituição de nomes por códigos), ou **anonimização** (remoção de qualquer informação que possa identificar o titular), sempre que possível, para reduzir o risco.
  - *Opcionalmente, o sistema pode oferecer uma função de anonimização automática para relatórios e pesquisas internas.*
- **Direito do Titular dos Dados (Data Subject Rights):** O CRM deve ter funcionalidades que permitam à consultoria atender às requisições dos titulares, conforme a LGPD:
  - **Direito de Acesso e Retificação:** Fácil acesso e correção dos dados cadastrais pelo cliente.
  - **Direito ao Esquecimento/Eliminação:** Capacidade de **excluir permanentemente** todos os dados de um cliente (PF) de todos os DBs (*multi-tenancy* e *caching*), quando solicitado e permitido legalmente.
- **Encarregado de Dados (DPO) e Registro de Consentimento:** O CRM deve registrar a **base legal** para o tratamento de cada dado pessoal, incluindo o consentimento (se aplicável), conforme exigido pela LGPD.

---

### 3. Isolamento de Dados (*Multi-tenancy Seguro*)

Conforme discutimos, a segurança do modelo *multi-tenancy* é a chave para a comercialização.

- **Segregação Lógica no DB Relacional:** No **PostgreSQL** (DB do CRM/Workflow), cada tabela crítica deve incluir uma coluna **tenant\_id**.
  - Todas as consultas SQL feitas por qualquer microserviço devem **obrigatoriamente** incluir o filtro WHERE `tenant_id = [ID da Consultoria Logada]`.
  - Isso garante que, mesmo que ocorra um erro de lógica de aplicação, o sistema **fisicamente não acessará** os dados de outro cliente.

- **Isolamento no Banco de Busca (Elasticsearch):** O Elasticsearch também deve usar o `tenant_id` em seus índices ou filtros. A pesquisa semântica da Consultoria A só pode retornar documentos indexados que pertençam à Consultoria A.
- **Gateway de API (API Gateway):** Um microserviço dedicado atuará como **ponto de entrada único** para todas as requisições externas. Ele é responsável por:
  1. Autenticar o usuário.
  2. Verificar as permissões (RBAC).
  3. Associar o **tenant\_id** à requisição antes de enviá-la ao microserviço correto.

Essa abordagem garante que a segurança seja tratada como parte integrante do design.

---

### *Pilha Tecnológica (Tech Stack)*

---

Definir a **Pilha Tecnológica (Tech Stack)** é a etapa final de planejamento. Ela deve ser robusta, flexível e moderna, otimizada para a CLOUD e para as necessidades de processamento de IA/PLN.

Aqui está a sugestão da pilha tecnológica para o seu SaaS de consultoria jurídica:

---

#### **Pilha Tecnológica Otimizada para SaaS Jurídico com IA**

##### 1. **Backend e Microserviços (A Espinha Dorsal)**

Para a construção dos microserviços, a prioridade é a performance e a compatibilidade com a CLOUD.

| Componente          | Opção Recomendada | Por Que?                                                                                                                                                       |
|---------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Linguagem Principal | Python            | É a linguagem dominante para a IA e <i>Machine Learning</i> (Módulos Preditivo e Semântico). Além disso, possui <i>frameworks</i> robustos para microserviços. |



| Componente     | Opção Recomendada                                  | Por Que?                                                                                                                                                                         |
|----------------|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frameworks Web | FastAPI ou Django/Flask                            | FastAPI é ideal para microsserviços por ser leve, rápido e ter suporte nativo a operações assíncronas, essencial para o desempenho na CLOUD.                                     |
| Comunicação    | REST APIs (Síncrona) e Kafka/RabbitMQ (Assíncrona) | O uso de filas de mensagens (como Kafka) é crucial para o <i>workflow</i> e para a IA garantindo que tarefas longas (ex: treinar um modelo) não bloqueiem o restante do sistema. |
| Orquestração   | Kubernetes (K8s)                                   | Essencial para o gerenciamento automático dos <i>containers</i> Docker e para a escalabilidade horizontal dos microsserviços.                                                    |

## 2. 🧠 Inteligência Artificial (O Diferencial)

A IA será implementada principalmente em microsserviços dedicados, usando as seguintes bibliotecas:

| Funcionalidade                                | Biblioteca Principal              | Papel no Software                                                                                                                                                 |
|-----------------------------------------------|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Processamento de Linguagem Natural (PLN)      | spaCy e NLTK                      | Usadas para tokenização, extração de entidades (nomes, datas, locais) e compreensão sintática em documentos jurídicos.                                            |
| Pesquisa Semântica e <i>Embeddings</i>        | Hugging Face e Gensim             | Treinamento de modelos de linguagem específicos (como BERT ou modelos otimizados para o português jurídico) para a busca baseada em significado.                  |
| Análise Preditiva ( <i>Machine Learning</i> ) | Scikit-learn e PyTorch/TensorFlow | Implementação e treinamento dos algoritmos de classificação (regressão logística, florestas aleatórias, etc.) que preveem os resultados dos casos.                |
| MLOps (Operações de ML)                       | MLflow ou DVC                     | Gerenciamento do ciclo de vida dos modelos de IA garantindo que eles sejam rastreáveis, testáveis e possam ser atualizados facilmente (vitais para a manutenção). |

## 3. 💻 Frontend (Interface do Usuário)

O frontend precisa ser rápido e responsivo para oferecer uma boa experiência de usuário (UX).

| Componente    | Opção Recomendada | Por Que?                                                                                                                                                                                                           |
|---------------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Framework Web | React ou Vue.js   | São os <i>frameworks</i> JavaScript mais populares, oferecendo alta performance, construção modular de componentes (ideal para um SaaS) e facilidade na criação de interfaces complexas (como <i>dashboards</i> ). |
| Linguagem     | TypeScript        | Ajuda a detectar erros de programação mais cedo, aumentando a qualidade e a manutenibilidade do código do frontend.                                                                                                |

## Resumo e Considerações Finais

A escolha por **Python** no *backend* cria uma sinergia poderosa, pois a mesma linguagem usada para o desenvolvimento web é usada para a IA, simplificando a integração e a contratação de talentos.

O uso de **Kubernetes e Docker** é o que garante que esta pilha tecnológica possa ser **comercializada** de forma eficiente: você pode empacotar os microsserviços de Python (IA) e o servidor web (FastAPI) em containers e implantá-los em qualquer provedor CLOUD de forma escalável e com baixo custo por *tenant*.

---

### Modelo físico

---

Modelo físico de dados simplificado para o módulo **CRM (Customer Relationship Management)**, otimizado para a arquitetura **multi-tenancy** (múltiplos clientes/consultorias) e para o **PostgreSQL**.

Este modelo garante a **segurança de dados** e a **rastreabilidade** essenciais para a LGPD e para a operação da consultoria.



#### Modelo Físico de Dados para o Módulo CRM (PostgreSQL)

O coração do modelo é a tabela *tenant*, que define o isolamento de dados, e a tabela *lead*, que incorpora o campo para o *Lead Scoring* da IA.

---

## 1. Tabelas de Estrutura e Isolamento

| Tabela         | Coluna       | Tipo de Dados | Restrições       | Descrição                                                 |
|----------------|--------------|---------------|------------------|-----------------------------------------------------------|
| <b>tenant</b>  | tenant_id    | SERIAL        | PK, NOT NULL     | <b>ID da Consultoria Jurídica</b> (Chave de Isolamento).  |
|                | nome         | VARCHAR(255)  | NOT NULL         | Nome da Consultoria.                                      |
|                | data_criacao | TIMESTAMP     | DEFAULT NOW()    | Data de criação do <i>tenant</i> .                        |
| <b>usuario</b> | usuario_id   | SERIAL        | PK, NOT NULL     | ID do Consultor/Usuário interno.                          |
|                | tenant_id    | INTEGER       | FK (tenant)      | <b>Isolamento:</b> A que consultoria o usuário pertence.  |
|                | email        | VARCHAR(255)  | UNIQUE, NOT NULL | Login e contato do usuário.                               |
|                | perfil       | VARCHAR(50)   | NOT NULL         | Nível de acesso (Ex: 'ADMIN', 'CONSULTOR', 'ESTAGIARIO'). |

## 2. Tabelas de *Leads* e Clientes

Esta seção rastreia o ciclo de vida, desde o primeiro contato até a conversão.

| Tabela      | Coluna            | Tipo de Dados | Restrições            | Descrição                                       |
|-------------|-------------------|---------------|-----------------------|-------------------------------------------------|
| <b>lead</b> | lead_id           | SERIAL        | PK, NOT NULL          | ID exclusivo do <i>Lead</i> .                   |
|             | tenant_id         | INTEGER       | FK (tenant), NOT NULL | <b>Isolamento:</b> Pertence a qual consultoria. |
|             | nome_contato      | VARCHAR(255)  | NOT NULL              | Nome da pessoa de contato.                      |
|             | tipo_cliente      | VARCHAR(50)   | NOT NULL              | PF / PJ / Órgão Público.                        |
|             | assunto_interesse | VARCHAR(100)  |                       | Área jurídica de interesse (Ex:                 |

| Tabela         | Coluna          | Tipo de Dados | Restrições            | Descrição                                                           |
|----------------|-----------------|---------------|-----------------------|---------------------------------------------------------------------|
|                |                 |               |                       | Trabalhista, Tributário).                                           |
|                | <b>score_ia</b> | NUMERIC(5, 2) |                       | <b>Pontuação de 0 a 100</b> gerada pela IA ( <i>Lead Scoring</i> ). |
|                | data_conversao  | DATE          |                       | Data em que o <i>lead</i> se tornou cliente (ou NULL).              |
| <b>cliente</b> | cliente_id      | SERIAL        | PK, NOT NULL          | ID do cliente efetivo.                                              |
|                | tenant_id       | INTEGER       | FK (tenant), NOT NULL | <b>Isolamento.</b>                                                  |
|                | lead_id         | INTEGER       | FK (lead), UNIQUE     | Liga o cliente ao seu histórico de <i>lead</i> .                    |
|                | cpf_cnpj        | VARCHAR(20)   | UNIQUE, NOT NULL      | Documento de identificação.                                         |
|                | segmento        | VARCHAR(100)  |                       | Sector de atuação (para PJ).                                        |

### 3. Tabelas de Interação e Comunicação

Usadas para registrar o *engagement* do *lead* e do cliente, alimentando o treinamento do *Lead Scoring*.

| Tabela           | Coluna          | Tipo de Dados | Restrições            | Descrição                          |
|------------------|-----------------|---------------|-----------------------|------------------------------------|
| <b>interacao</b> | interacao_id    | SERIAL        | PK, NOT NULL          | ID da interação.                   |
|                  | tenant_id       | INTEGER       | FK (tenant), NOT NULL | <b>Isolamento.</b>                 |
|                  | lead_cliente_id | INTEGER       | NOT NULL              | ID do lead ou cliente relacionado. |

| Tabela          | Coluna          | Tipo de Dados  | Restrições            | Descrição                               |
|-----------------|-----------------|----------------|-----------------------|-----------------------------------------|
|                 | tipo_interacao  | VARCHAR(50)    | NOT NULL              | Ex: 'EMAIL', 'LIGACAO', 'REUNIAO'.      |
|                 | data_hora       | TIMESTAMP      | NOT NULL              | Quando a interação ocorreu.             |
|                 | resumo          | TEXT           |                       | Descrição ou anotações da interação.    |
| <b>proposta</b> | proposta_id     | SERIAL         | PK, NOT NULL          | ID da proposta comercial.               |
|                 | tenant_id       | INTEGER        | FK (tenant), NOT NULL | <b>Isolamento.</b>                      |
|                 | lead_id         | INTEGER        | FK (lead), NOT NULL   | Proposta gerada para qual <i>lead</i> . |
|                 | valor_total     | NUMERIC(10, 2) |                       | Valor da proposta.                      |
|                 | status_proposta | VARCHAR(50)    | NOT NULL              | Ex: 'PENDENTE', 'ACEITA', 'RECUSADA'.   |

### 🔑 Chave de Implementação: O tenant\_id

Observe que o campo **tenant\_id** aparece em todas as tabelas principais. Este é o mecanismo físico para o isolamento de dados no PostgreSQL:

- Todas as **consultas SELECT, UPDATE ou DELETE** em qualquer microserviço devem incluir WHERE tenant\_id = [ID da Consultoria Logada].

Este modelo é uma excelente base para o MVP do seu CRM. Ele suporta a separação de clientes, a rastreabilidade do *pipeline* e já inclui o campo score\_ia necessário para o seu módulo de Inteligência Artificial.

A estrutura de dados apresentada **não garante** a Autenticação de Múltiplos Fatores (**MFA** ou 2FA) por si só. A autenticação de usuário é uma **funcionalidade de segurança** implementada na **camada de aplicação** e no **API Gateway**, e não no banco de dados.

No entanto, a tabela usuario no PostgreSQL precisa de **campos adicionais** para armazenar e gerenciar o MFA.

## Implementação da Autenticação de Múltiplos Fatores (MFA)

Para implementar o MFA no seu MVP, você precisará de modificações tanto na **estrutura de dados** (para rastrear o *status* do MFA) quanto no **microsserviço de autenticação** (para aplicar a lógica).

### 1. Modificações na Estrutura de Dados (usuario - PostgreSQL)

A tabela `usuario` deve ser atualizada para armazenar as configurações necessárias para o MFA, que geralmente envolve o uso de aplicativos autenticadores (como Google Authenticator) baseados em TOTP (Time-based One-Time Password).

| Tabela  | Coluna       | Tipo de Dados | Restrições    | Descrição                                                                       |
|---------|--------------|---------------|---------------|---------------------------------------------------------------------------------|
| usuario | mfa_ativo    | BOOLEAN       | DEFAULT FALSE | Indica se o MFA está ativado para este usuário.                                 |
|         | mfa_secret   | VARCHAR(255)  | NULL          | Chave secreta criptografada (gerada no registro) para o TOTP.                   |
|         | backup_codes | TEXT          | NULL          | Códigos únicos para recuperação de acesso (armazenados de forma criptografada). |

Exportar para as Planilhas

### 2. Lógica de Autenticação (Microsserviço de Segurança/API Gateway)

A lógica real do MFA reside no *backend*. O microsserviço de segurança (que você implementará em Python/FastAPI) será responsável por:

- **Registro (Primeiro Fator):** O usuário fornece **e-mail e senha** (armazenada de forma *hash*).
- **Registro de MFA (Segundo Fator):** Se o usuário ativar o MFA, o sistema:
  1. Gera a **chave secreta** (`mfa_secret`).
  2. Exibe o QR code ao usuário.
  3. Verifica o primeiro código TOTP fornecido pelo usuário para confirmar a sincronização.
  4. Define `mfa_ativo = TRUE` no banco de dados.
- **Login com MFA:**
  1. O usuário insere e-mail e senha (Verificação do **Primeiro Fator**).

2. O sistema verifica se `mfa_ativo` é `TRUE`.
3. Se `TRUE`, o sistema solicita o **código TOTP** (Verificação do **Segundo Fator**).
4. O microsserviço usa a biblioteca de TOTP (como `pyotp` em Python) e a chave secreta (`mfa_secret`) para validar o código fornecido.

A arquitetura de **Microsserviços/API Gateway** é ideal para isso, pois o *gateway* pode ser configurado para ser o único ponto que exige a conclusão bem-sucedida de ambos os fatores antes de conceder acesso aos microsserviços internos (CRM, Workflow, etc.).